DS 642: Applications of Parallel Computing

Shahriar Afkhami

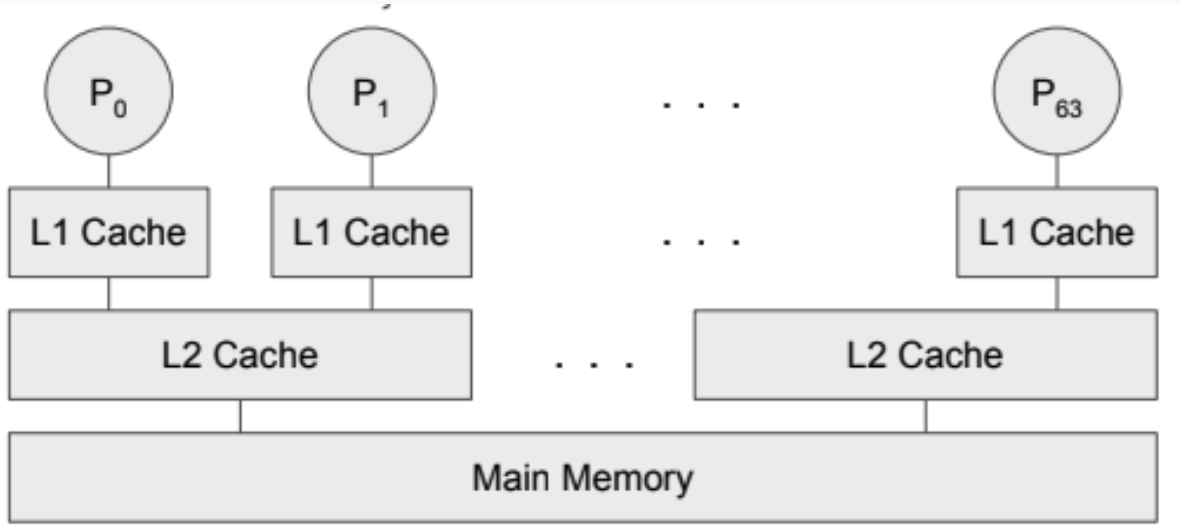
Week 4:
Shared Memory Programming with OpenMP

Parallel Programming Models

Programming model is made up of the languages and libraries that create an abstract view of the machine

- Control
  - How is parallelism created?
- Data
  - What is private vs shared data?
  - How is logically shared data accessed or communicated?
- Synchronization
  - What operations can be used to coordinate parallelism?
  - What are the atomic (indivisible) operations?
- Cost
  - How do we account for the cost of each of the above?

Shared memory vs message passing



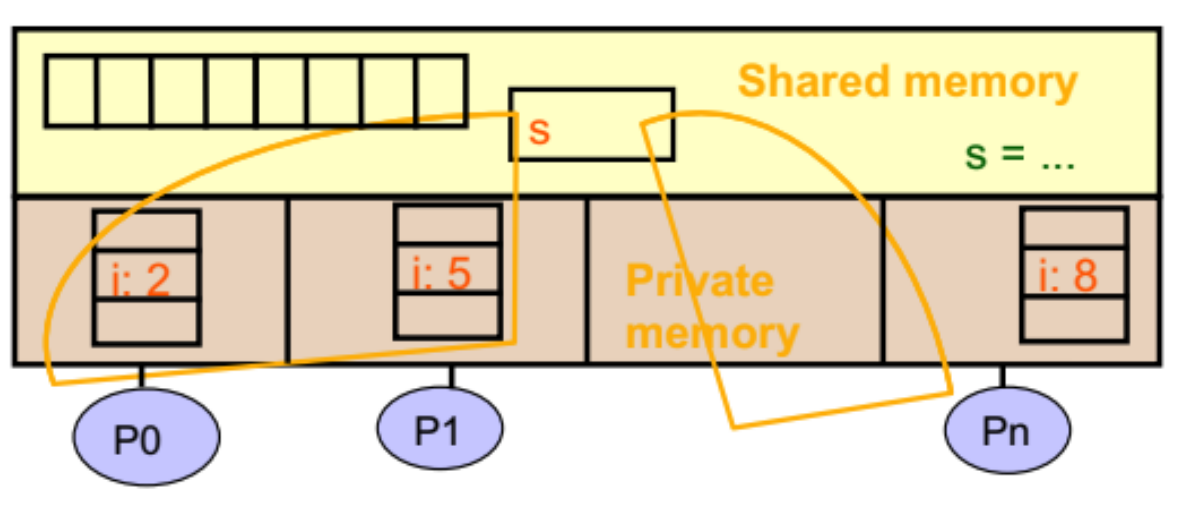
- Implicit communication via memory vs explicit messages
- Threads communicate + synchronize with shared variables

Simple example: Consider the following:

$$\sum_{i=0}^{n-1} f(A[i])$$

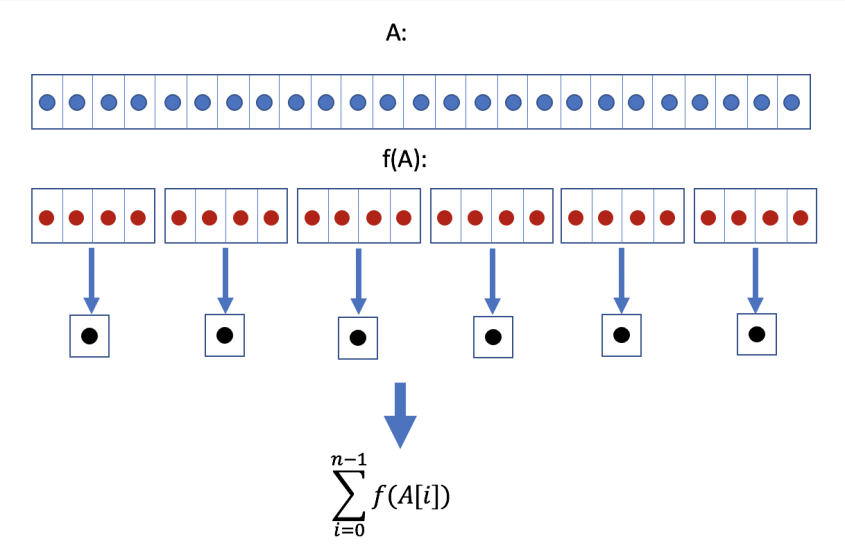
Where does A live? What work will be done by each processor? How the single result is obtained?

Shared Memory Programming Model



- Program is a collection of threads of control.
- Each thread has a set of private variables, e.g., local stack variables.
- Also a set of shared variables, e.g., static variables, shared common blocks, or global heap.
  - Threads communicate implicitly by writing and reading shared variables.
  - Threads coordinate by synchronizing on shared variables

Shared memory strategy



- small number of processors $p \ll n = size(A)$, attached to a single memory.
- parallel decomposition: each evaluation and partial sum is a task.
- assign n/p numbers to each p.
  - each compute independent private results and partial sums.
  - collect the partial sums and compute global sum.

Data Race Example

Problem is a race condition on variable s in the program:

```
static int s = 0;
```

Processor 1:

```
for i = 0, n/2 - 1;
    s = s + f(A[i]);
```

Processor 2:

```
for i = n/2, n - 1;
    s = s + f(A[i]);
```

- A race condition or data race occurs when:
  - two processors (or two threads) access the same variable, and at least one does a write.
  - The accesses are concurrent (not synchronized) so they could happen simultaneously.

Assume $A = [3, 5]$, with $f(x) = x^2$, what do you think that the result of the above code can be?

OpenMP programming model

Program consists of *threads* of control.

- Can be created dynamically
- Each has private variables (e.g. local)
- Each has shared variables (e.g. heap)
- Communication through shared variables
- Coordinate by synchronizing on variables

Introduction to OpenMP

- What is OpenMP?
  - Open specification for Multi-Processing
  - “Standard” application programming interface (API) for defining multi-threaded shared-memory programs
  - openmp.org
- High-level API
  - Preprocessor (compiler) directives ($\sim$ 80%)

    `#pragma omp` *construct* `[`*clause* `...]`
  - Library Calls ($\sim$ 19%)

    `#include <omp.h>`
  - Environment Variables ($\sim$ 1%)

A Programmer's View of OpenMP

- OpenMP is a portable, threaded, shared-memory programming specification with "light" syntax
  - Requires compiler support (C or Fortran)
- OpenMP will:
  - Allow a programmer to separate a program into serial regions and parallel regions, rather than T concurrently-executing threads.
  - Hide stack management.
  - Provide synchronization constructs.
- OpenMP will not:
  - Parallelize automatically
  - Guarantee speedup
  - Provide freedom from data races

Compiling OpenMP Programs

A practical aside...

- Intel has OpenMP support by default

```
icc -c -qopenmp foo.c
icc -o -qopenmp my_code.exe foo.o
```

- GCC has OpenMP support by default

```
gcc -c -fopenmp foo.c
gcc -o -fopenmp my_code.exe foo.o
```

- LLVM has OpenMP support by default (more recent)

```
clang -c -fopenmp foo.c
clang -o -fopenmp my_code.exe foo.o
```

- Apple LLVM *does not* support OpenMP
  - And `gcc` on OS X now aliases `clang`
  - Can use Hombrew for alternate compiler

Parallel “hello world”

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main()
5  {
6      #pragma omp parallel // Do this part in parallel
7      printf("Hello world from %d\n",
8             omp_get_thread_num());
9
10     return 0;
11 }
```

Parallel “hello world”

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main()
5  {
6      omp_set_num_threads(16);
7
8      #pragma omp parallel // Do this part in parallel
9      printf("Hello world from %d\n",
10             omp_get_thread_num());
11
12     return 0;
13 }
```

The OpenMP Common Core

Most OpenMP programs only use these

OpenMP pragma, function, or clause	Concepts
#pragma omp parallel	Parallel region, teams of threads, structured block, interleaved execution across threads
int omp_get_thread_num() int omp_get_num_threads()	Create threads with a parallel region and split up the work using the number of threads and thread ID
double omp_get_wtime()	Speedup and Amdahl's law. False Sharing and other performance issues
setenv OMP_NUM_THREADS N	Internal control variables. Setting the default number of threads with an environment variable
#pragma omp barrier #pragma omp critical	Synchronization and race conditions. Revisit interleaved execution.
#pragma omp for #pragma omp parallel for	Worksharing, parallel loops, loop carried dependencies
reduction(op:list)	Reductions of values across a team of threads
schedule(dynamic [,chunk]) schedule (static [,chunk])	Loop schedules, loop overheads and load balance
private(list), firstprivate(list), shared(list)	Data environment
nowait	Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive)
#pragma omp single	Workshare with a single thread
#pragma omp task #pragma omp taskwait	Tasks including the data environment for tasks.

Data scoping: Shared and private

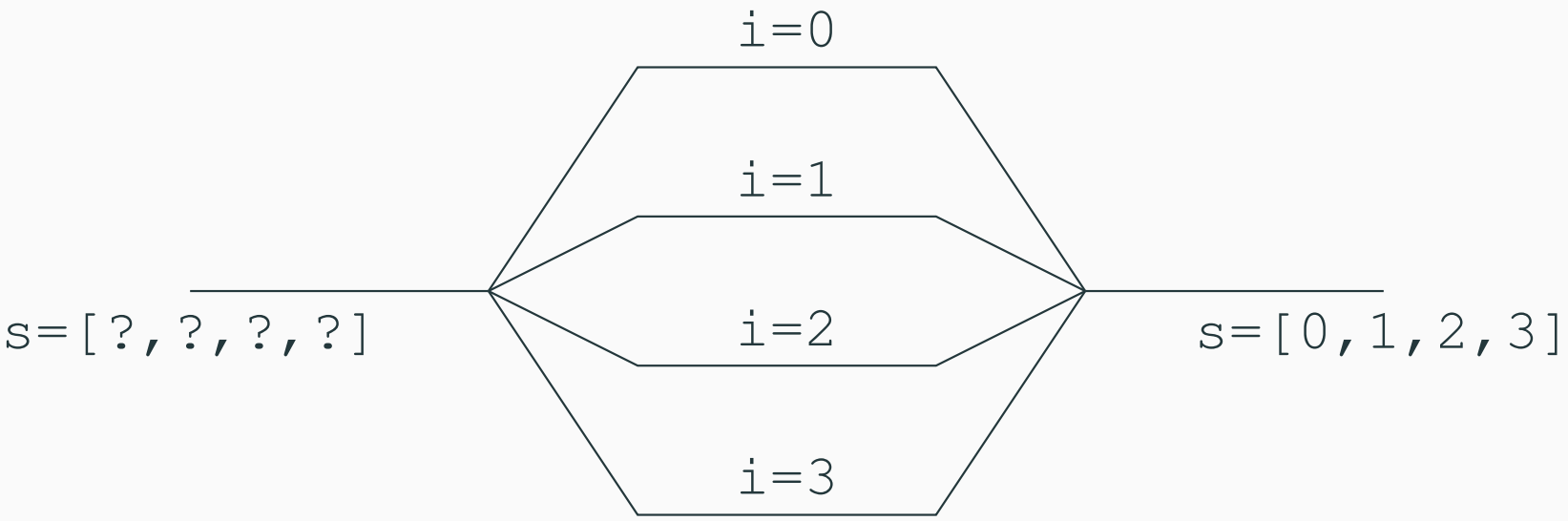
`#pragma omp` *construct* `[`*clause* `...]`

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main()
5  {
6  ...
7
8  #pragma omp parallel private(x)
9  {
10 ...
11 }
12 }
```

Annotations distinguish between different types of sharing:

- `shared(x)` (default): One `x` shared everywhere
- `private(x)`: Thread gets own `x` (indep. of master)

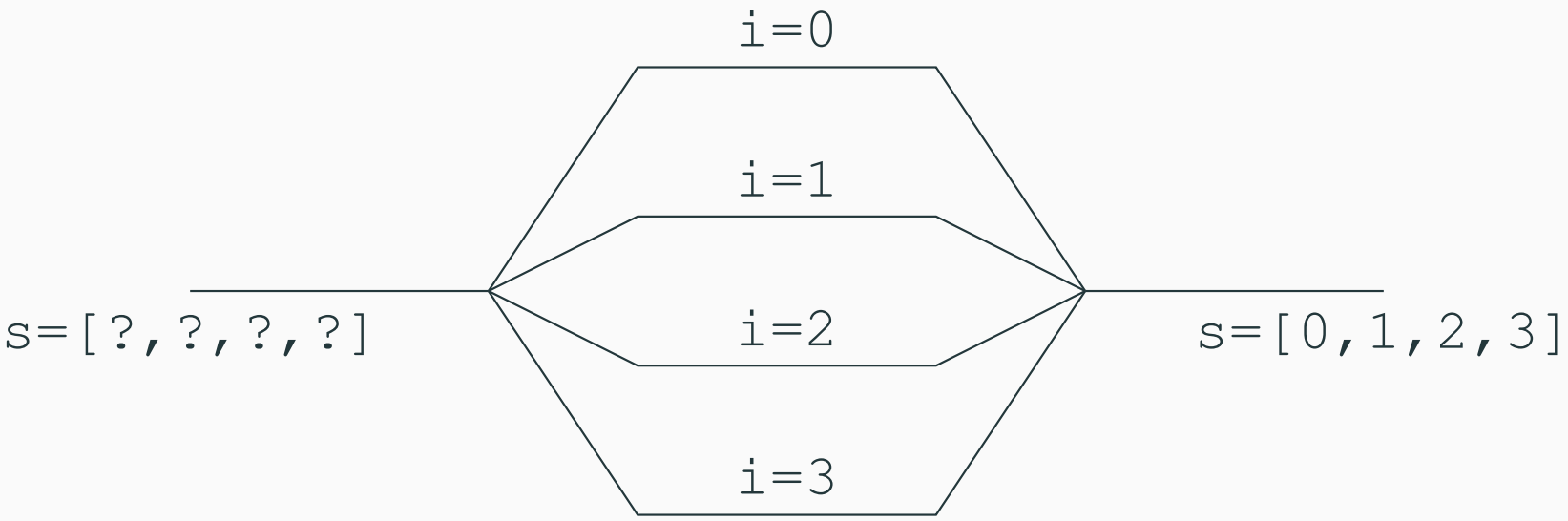
Parallel regions



Parallel region

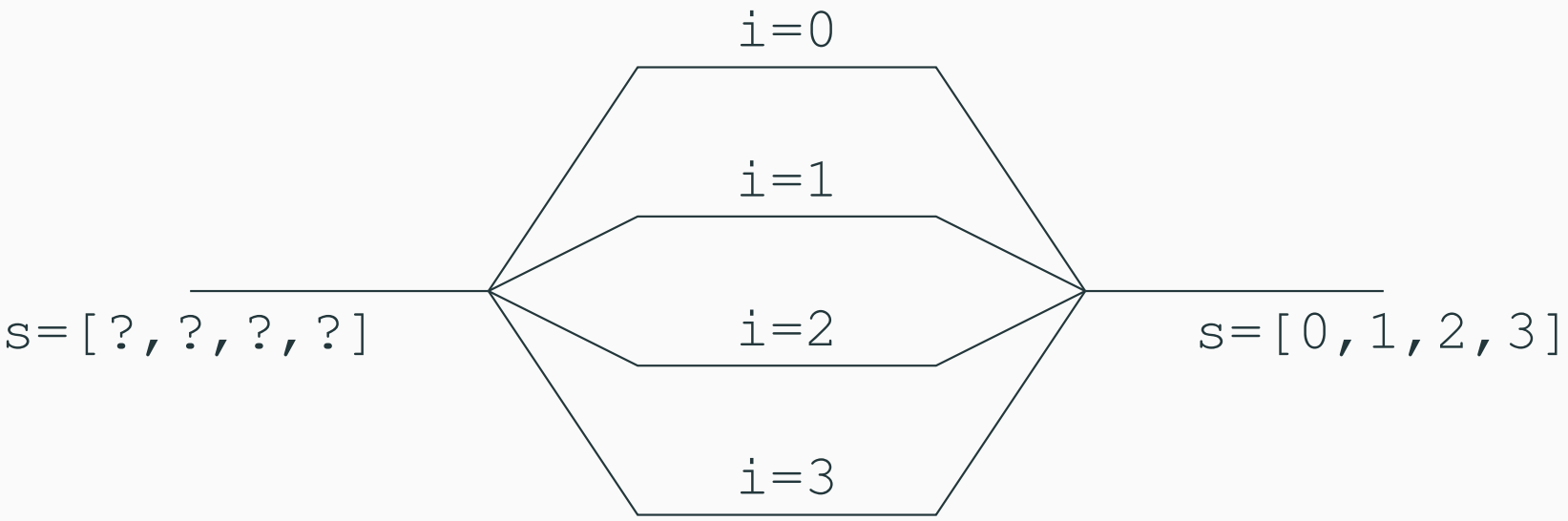
- Basic model: fork-join
- Each thread runs same code block
- Annotations distinguish shared (*s*) and private (*i*) data
- *Relaxed consistency* for shared data

Parallel regions




```
1  double s[MAX_THREADS];
2  int i;
3  #pragma omp parallel shared(s) private(i)
4  {
5    i = omp_get_thread_num();
6    s[i] = i;
7  }
8  // Implicit barrier here
```

Parallel regions




```
1  double s[MAX_THREADS];  // default shared
2  #pragma omp parallel
3  {
4    int i = omp_get_thread_num();  // local, so private
5    s[i] = i;
6  }
7  // Implicit barrier here
```

Parallel regions

Several ways to control num threads

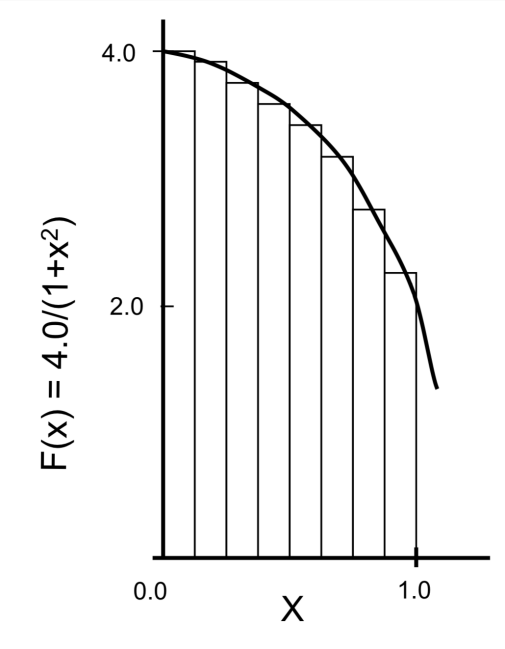
- Default: System chooses
- Environment: `export OMP_NUM_THREADS=4`
- Function call: `omp_set_num_threads(4)`
- Clause: `#pragma omp parallel num_threads(4)`

Can also nest parallel regions.

Parallel “hello world”

```
1  #include <stdio.h>
2  #include <omp.h>
3  int main(int argc, char *argv[])
4  {
5      int nthreads, id;
6      omp_set_num_threads(4);
7
8      #pragma omp parallel private(nthreads, id)
9      {
10     id = omp_get_thread_num(); \\private
11     printf("Hello world from %d\n", id);
12
13     if (id == 0) {
14         nthreads = omp_get_num_threads();
15         printf("Number of threads = %d\n", nthreads);
16     }
17     }
18     return 0;
19 }
```

Example: Numerical Integration



$$\int_0^1 \frac{4}{1+x^2}\,dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^{n} F(x_i)\Delta x \approx \pi$$

Serial π program

```
1  #include <stdlib.h>           /* Standard Library */
2  #include <stdio.h>            /* IO library */
3  #include <math.h>             /* Math librry */
4  #define pi_true (acos(-1.))   /* double pi_true = 3.14159265358
5
6  int main(int argc, char* argv[]) {
7
8    double h, x, pi=0.; int i; long int n = 100000;
9
10   h = 1. / (double) n; /* Set subinterval size */
11
12   for (i = 0; i < n; i++) {   /* Perform integration */
13     x = h * (i + .5);
14     pi += 4.*h/(1. + x*x);
15   }
16   printf(" computed pi = %.16e error = %.16e\n",
17            pi, pi_true - pi);
18   return(0);
19 }
```

Example: Numerical Integration (OpenMP)

```c
#include <stdio.h>
#include <math.h>
#include <omp.h>

#define NUM_THREADS 8
static long n = 100000000;

int main ()
{
  int i, nthreads;
  double h, pi, sum[NUM_THREADS] = {0.}, t;
  h = 1./(double) n;

  omp_set_num_threads(NUM_THREADS);

  t = omp_get_wtime();
#pragma omp parallel
{
  double x;
  int id, i, nthrds;
  id = omp_get_thread_num();
  nthrds = omp_get_num_threads();
  if (id == 0) nthreads = nthrds;
  for (i = id; i < n; i = i + nthrds) {
    x = h*(i + .5);
    sum[id] += 4./(1. + x*x);
   }
}
  for(i = 0, pi = 0.; i < nthreads; i++)
  pi += sum[i]*h;
  t = omp_get_wtime() - t;
  printf("\n Computed pi is %.16e with %d threads and error %.16e computed in %f secs \n ",
          pi, nthreads, acos(-1.) - pi, t);
return(0);
}
```

OpenMP Summary So Far

OpenMP is a compiler-based technique to create concurrent code from (mostly) serial code

Next time:

- OpenMP can enable (easy) parallelization of loop-based code
- Task-based parallelism
- Other parallel work divisions

More Information:
`https://hpc-tutorials.llnl.gov/openmp/`